THE NATIONAL UNIVERSITY OF LESOTHO
Department of Mathematics and Computer Science · Faculty of Science and Technology
CS3520 Computer Organisation and Architecture · AY 2026/2027

LIMO Group Project • Milestone M1 Submission

LIMO Instruction Set Architecture

		#	Member	Student no.	GitHub
Team	The Control Unit				
Lab group	A, B, C	1.	Lerato Molapo	202400966	leratomolapo254
Release tag	m1	2.	Bulane Motsoahae	202400040	s202400040
Commit	*[short SHA]*	3.	Thaabe Pheko	202404092	phekothaabe2-blip
Due	Wed 14 Oct 2026, 23:59	4.	Thabang Sekoqo	202321323	thabangsekoqo
Page limit	6 pages	5.	Puleng Mohlomi	202400345	Puleng-Mohlomi
		6.	Poloko Pompo	202322766	Poloko-Praise-Pompo
		7.	Ts'eliso Metsing	202321242	Metsing204

Repository https://github.com/s202400040/limo-The-Control-Unit -
Live site https://s202400040.github.io/limo-The-Control-Unit-/
Assessed Part I: ISA specification A1–A5 (40) and design-decision log A6 (20); defended individually at D1 (40).

1 Team Charter and Set-up

Project board	https://github.com/s202400040/limo-The-Control-Unit-/projects/1/views/4
Meeting rhythm	Stand-up Tue/Thu 19:00. Sprint planning Mon 19:00 Sprint review Sunday 15:00 before deadline.
Decision rule	*Majority vote. Escalate to coordinator if not resolved within 24hrs*
Definition of done	Code merged to main through PR passes. Issue moved to done on project board.
Branch and PR rules	Main is protected, no direct push. PR needs 1 reviewer and 1 approval.
When a member is absent or not contributing	Tag in project board and message on whatsapp. Re-assign task to available member, if no response or 2 consecutive sprints missed, group meets without member and escalates to coordinator with evidence (commits and board activity)

2 ISA Specification

2.1 Overview

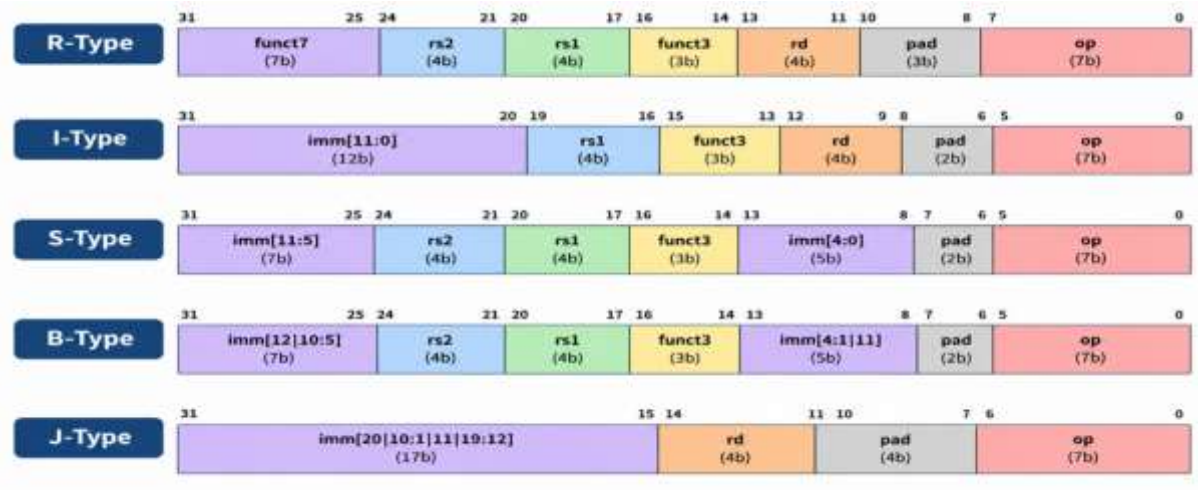
	Specification	Rationale
Architecture style	32-bit load–store	Only loads/stores access data memory.
Register bank	13 GPRs and 3SPRs	4-bit register fields; r0 is zero.
Addressing modes	Register, immediate, base+offset ,PC-relative	Supports ALU, memory and control flow.
Memory,	Load-store	Only loads and stores access data memory.
endianness,	Little-endian	Least-significant byte occupies the lowest address.
Alignment	word-aligned	Word accesses use addresses divisible by 4.
Instruction	Fixed 32 bits	Simplifies instruction fetch and pipeline control.
formats	R, I, S, B; J optional	Covers required operations and jump support.
Departures from	16 registers; 4-bit identifiers	Differs from RV32I's 32-register bank
RV32I		

2.2 Register File

Number	Sesotho name	Alias	Type	Purpose and convention
r0	lefela	zero	SPR	Hard-wired to 0; writes are ignored
r2	boela	ra	SPR	Return address
r1	sepache	sp	SPR	Stack pointer
r3·r6	khang0·3	args	GPR	General purpose (arguments)
r7·r11	nakoana0·4	tmp	GPR	General purpose (temporaries)
r12·r15	boloka0·3	saved	GPR	General purpose (saved)
PC	*sebali_sa_lenaneo*	–	SPR	Address of the current instruction

a. Instruction Formats

> *The RV32I field widths are shown as a starting point. Change them to suit your register-file size and state where any freed bits go. Add U or J only if you use them.*



Immediate ranges: I: -2048 to +2047; S: -2048 to +2047; B: -4096 to +4094

b. Instruction Set and Sesotho Mnemonics

Class	Mnemonic	Meaning	Syntax	Operation	Fmt	RV32I
Arithmetic	eketsa	add; increase	`eketsa rd, rs1, rs2`	R[rd] → R[rs1] + R[rs2]	R	add
Arithmetic	fokotsa	subtract	*fokotsa rd, rs1, rs2*	*rd = rs1 − rs2*	*R*	*sub*
Arithmetic	eketsi	add immediate	eketsi rd, rs1, imm	R[rd] · R[rs1] + imm	I	addi
Logic	le	AND	le rd, rs1, rs2	R[rd] · R[rs1] AND R[rs2]	R	and
Logic	kapa	OR	kapa rd, rs1, rs2	R[rd] · R[rs1] OR R[rs2]	R	or
Logic	suthela	shift left logical	suthela rd, rs1, rs2	R[rd] · R[rs1] << R[rs2][4:0]	R	sll
Memory	bala	load word	bala rd, imm(rs1)	R[rd] · Mem[R[rs1]+imm]	I	lw
Memory	boloka	store word	boloka rs2, imm(rs1)	Mem[R[rs1]+imm] · R[rs2]	S	sw
Branch	lekana	branch if equal	lekana rs1, rs2, label	if equal: PC · PC+offset	B	beq
Branch	selekane	branch if not equal	selekane rs1, rs2, label	if unequal: PC · PC+offset	B	bne

.

Lexical rules. The assembler accepts the ASCII apostrophe (') inside a mnemonic token so future Sesotho mnemonics can represent forms such as ts' or ch'. Curly Unicode apostrophes are rejected to keep tokenisation predictable on a standard keyboard.

c. Encoding Worked Examples

Instruction	Fmt	Field values	Binary (bit 31 … 0)	Hex
eketsa r7, r8, r9	R	*Opcode:0110011* *Pad:000* *rd:0111* *funct3:000* *rs1:1000* *rs2:1001* *Funct7:0000000*	*0000 0001 0011 0000 0001 1100 0011 0011*	*0x01301C33*
eketsi r10, r7, 10	I	*opcode:0010011* *pad:000* *rd:1010* *funct3:000* *rs1:0111* *imm:000000001010*	*0000 0000 1010 0111 0001 0100 0001 0011*	*0x00A71413*
boloka r10, 8(r7)	S	*opcode:0100011* *pad:00* *imm:01000* *funct3:010* *rs1:0111* *rs2:1010*	*0000 0001 0100 1110 1001 0000 0010 0011*	*0x014E9023*
lekana r7, r8, +8	B	*imm:0000000* *opcode:1100011* *pad:00* *imm:01000* *funct3:010* *rs1:1000* *rs2:0111* *imm:0000000*	*0000 0000 1111 0000 1001 0000 0110 0011*	*0x00F09063*

3 Design-Decision Log

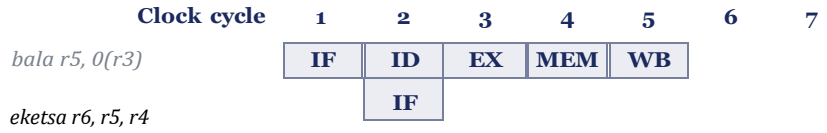
(a) Register-file size. What does it cost and buy in encoding bits, forwarding-comparator width and register pressure?
Sixteen registers give 4-bit register fields, reducing encoding and comparator width compared with 32 registers. The cost is greater register pressure: longer programs may need more register reuse or memory spills.

(b) Architectural or microarchitectural? Which registers are programmer-visible, and which exist only inside the pipeline (IR, IF/ID, …)? Why must the ISA not specify the latter?

r0–r15 and PC are architectural because programs can depend on them. IF/ID, ID/EX, EX/MEM and MEM/WB are pipeline implementation registers. They must not be part of the ISA because the ISA should describe programmer-visible behaviour independently of one pipeline implementation.

(c) Branch resolution. In which stage does LIMO resolve branches, and what does that cost in flushed instructions versus extra hardware?
Branches are resolved in EX. This fits the standard five-stage datapath and keeps comparison and target generation simple. A taken branch can require flushing younger instructions already in the pipeline. Resolving earlier could reduce flushes but would add hardware/control complexity in ID.

(d) Load-use. Why does a load followed by a dependent instruction still stall under full forwarding? Complete the diagram with your own two instructions.
A load's data becomes available after memory access, while the dependent instruction needs its operand in EX. Even with forwarding, the value is not available soon enough for the immediately following instruction. Therefore, a bubble is required.



(e) A flags register? RISC-V has none. Would LIMO benefit from one, and what would it do to your hazards?
LIMO does not use a flags register. BEQ and BNE compare registers directly. A flags register would introduce another architectural state and additional dependencies between an instruction that changes flags and a branch that consumes them.

(f) What breaks first as programs grow: immediate range, branch reach or register count?
Register pressure is likely to appear early in small programs, while immediate range and branch reach become more important as programs and loops grow.

4 Sample LIMO Programs

Program 1: Addition (it adds numbers)

```
eketsi r3, r0, 7        # r3 = 7
eketsi r4, r0, 5        # r4 = 5
eketsa r5, r3, r4       # r5 = r3 + r4 = 12
```

Expected final state: r5 = 12

Program 2: Store and Load Word (it stores and loads)

```
eketsi r3, r0, 100      # Base address = 100
eketsi r4, r0, 42       # Data value = 42
boloka r4, 0(r3)        # Memory[100] = 42
bala   r5, 0(r3)        # r5 = Memory[100] = 42
```

Expected final state: r5=memory[100]=42

Program 3: Simple Increment Loop (Loop incrementation)

```
eketsi r3, r0, 0    # Counter r3 = 0
eketsi r4, r0, 5    # Limit r4 = 5
loop:
    eketsi r3, r3, 1        # r3 = r3 + 1
    selekane r3, r4, loop   # Loop while r3 != r4
```

References

Patterson, D. A. and Hennesy, J.L. Computer Organisation and Design RISC-V Edition: The Hardware Software Interface. 2nd Ed. Morgan Kaufmann, 2020- Used for ISA format, immediate ranges, and control signal design.

AI-Use Declaration

Tool	Used for	How we checked or changed the output
Chatgpt(GPT 4o)	Github pages markdown table for ISA Spec	Checked markdown rendering locally before push. No ISA logic taken from AI.
Gemini 2.0	Brainstorming how to explain unused opcode spaces.	We rejected its suggestion to use 5b registers. Verified its explanation of immediate encoding against Patterson & Hennessy and RISC-V spec and recalculated the ranges -2048 to +2047 ourselves.

We confirm that every member can explain every part of this submission and the code behind it.

Sprint Note

Sprint goal: Complete M1

Member	Role this sprint	Delivered (issue and PR numbers)	PRs reviewed
1. Lerato Molapo	Hazard Unit Lead	#33, #16	# 16
2.Bulane Motsoahae	Interface Lead	#28, #11	# 11
3.Thaabe Pheko	ISA Architect	#32, #14	# 14
4.Thabang Sekoqo	Test Lead	#29, #12	# 12
5. Puleng Mohlomi	Scrum and Release Lead	#31, #13	# 13
6. Poloko Pompo	Assembler Lead	#34, #17	# 17
7. Ts'eliso Metsing	Pipeline Lead	#33, #15	# 15

Not done, and why**: none* ***Next sprint**: M2* ***Blockers: none

- □ Tag m1 pushed; link pasted on Google Classroom
- □ This PDF committed as docs/M1.pdf and uploaded
- □ \guidancefalse set; no [placeholders] left
- □ AI_USE.md updated in the repository
- □ Every member has merged or reviewed work this sprint
- □ Commit SHA above matches the tagged release